

do same for use case webserver hadoop

Here is the complete step-by-step procedure for the **Web Server HTTP Status Code Analysis (Hadoop MapReduce)** use case, structured in the exact same clear, terminal-based format as the previous lab exercises.

TXT

All system-dependent values (such as paths, directory names, script names, log files, user parameters, hostnames, and ports) that need to be updated when systems or use cases change are enclosed in `<< >>`.

Group A: System & HDFS Storage Operations (Linux / HDFS)

Terminal 1: Hadoop Cluster Initialization & File System Setup

1. Start Hadoop Services & Verify Processes:

Bash



```
start-dfs.sh
start-yarn.sh
jps
```

Troubleshooting Note: If YARN shows `Total Nodes: 0`, run the node list command, clear cache, check disk space, restart services, and verify again:

Bash



```
yarn node -list
rm -rf ~/.cache/*
df -h /
sudo du -xhd1 /var | sort -h
stop-yarn.sh
stop-dfs.sh
start-dfs.sh
```

```
start-yarn.sh  
jps
```

1. Create Local Working Directory:

Bash



```
cd ~  
mkdir -p <<web-log-mapreduce>>  
cd ~/<<web-log-mapreduce>>
```

1. Create Input Raw Web Log File:

Bash



```
nano <<web_logs.txt>>
```

Paste Sample Log Content:

Plaintext



```
192.168.1.10 GET /home 200  
192.168.1.11 GET /login 200  
192.168.1.12 GET /product 404  
192.168.1.13 GET /home 200  
192.168.1.14 GET /checkout 500  
192.168.1.15 GET /product 404  
192.168.1.16 GET /home 200  
192.168.1.17 GET /server 500
```

1. Verify Local Web Log File:

Bash



```
cat <<web_logs.txt>>
```

Group B: MapReduce Python Script Development & Local Testing

Terminal 1: Mapper and Reducer Implementation

1. Create Python Mapper Script:

Bash



```
nano <<StatusCodeMapper.py>>
```

Paste Mapper Code:

Python



```
#!/usr/bin/env python3

import sys

for line in sys.stdin:
    parts = line.strip().split()

    if len(parts) >= 4:
        status_code = parts[3]
        print(f"{status_code}\t1")
```

1. Make Mapper Executable & Perform Local Verification:

Bash



```
chmod +x <<StatusCodeMapper.py>>
cat <<web_logs.txt>> | python3 <<StatusCodeMapper.py>>
```

1. Create Python Reducer Script:

Bash



```
nano <<StatusCodeReducer.py>>
```

Paste Reducer Code:

Python



```
#!/usr/bin/env python3

import sys

current_code = None
current_count = 0

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue

    code, count = line.split('\t')
    try:
        count = int(count)
    except ValueError:
        continue

    if current_code == code:
        current_count += count
    else:
        if current_code:
            print(f"{current_code}\t{current_count}")
        current_code = code
        current_count = count

if current_code == code:
    print(f"{current_code}\t{current_count}")
```

1. Make Reducer Executable & Test Local MapReduce Pipeline:

Bash



```
chmod +x <<StatusCodeReducer.py>>  
cat <<web_logs.txt>> | python3 <<StatusCodeMapper.py>> | sort | python3 <<StatusCodeR
```

Expected Output:

Plaintext



200	4
404	2
500	2

1. Verify Project Files Directory:

Bash



```
ls -lh
```

(Ensure only `StatusCodeMapper.py`, `StatusCodeReducer.py`, and `web_logs.txt` are present).

Group C: Distributed HDFS Execution & Job Submission

Terminal 1: HDFS Directory Setup & Hadoop Streaming Job

1. Create Target HDFS Input Directory:

Bash



```
hdfs dfs -mkdir -p /user/$USER/<<web_logs>>/<<input>>
```

1. Upload Data File to HDFS:

Bash



```
hdfs dfs -put -f <<web_logs.txt>> /user/$USER/<<web_logs>>/<<input>>/
```

1. Verify Uploaded File in HDFS:

Bash



```
hdfs dfs -ls /user/$USER/<<web_logs>>/<<input>>
hdfs dfs -cat /user/$USER/<<web_logs>>/<<input>>/<<web_logs.txt>>
```

1. Remove Existing HDFS Output Directory (if any):

Bash



```
hdfs dfs -rm -r -f /user/$USER/<<web_logs>>/<<output>>
```

1. Execute Hadoop Streaming MapReduce Job:

Bash



```
hadoop jar /opt/hadoop/share/hadoop/tools/lib/<<hadoop-streaming-3.3.6.jar>> \
-input /user/$USER/<<web_logs>>/<<input>> \
-output /user/$USER/<<web_logs>>/<<output>> \
-mapper "python3 <<StatusCodeMapper.py>>" \
-reducer "python3 <<StatusCodeReducer.py>>" \
-file <<StatusCodeMapper.py>> \
-file <<StatusCodeReducer.py>>
```

1. Verify Final Hadoop Execution Output in HDFS:

Bash



```
hdfs dfs -ls /user/$USER/⟨⟨web_logs⟩⟩/⟨⟨output⟩⟩  
hdfs dfs -cat /user/$USER/⟨⟨web_logs⟩⟩/⟨⟨output⟩⟩/part-*
```

Expected Final Output:

Plaintext



200	4
404	2
500	2

Summary Flow for Exam Memory

Plaintext



```
start-dfs.sh / start-yarn.sh ↓ mkdir local dir ↓ nano web_logs.txt ↓ nano Mapper.py &
```